ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ВНУТРІШНІХ СПРАВ

Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №4

з дисципліни «Об’єктно-орієнтовне програмування»

Тема: «Множинне спадкування та композиція.»

Виконав:

курсант 2-го курсу

ННІ 4-25-203 інф

Кожевніков К.О.

Прийняв:

Доцент кафедри ІСТ, к.т.н.,
Слободянюк О.В.

КАМ’ЯНЕЦЬ-ПОДІЛЬСЬКИЙ – 2026

ВАРІАНТ 6. Медичний центр

Завдання:

Класи для роботи: Service, Appointment, Client. Створіть сервісний mixin для запису подій Appointment та композиційний компонент календаря. Усі записи вигадані.

КОД РОЗВ'ЯЗКУ

```
from abc import ABC, abstractmethod

class AuditableMixin:
    """Mixin-клас, що додає функціонал логування (аудиту) подій для запису."""

    def audit_record(self):
        """Повертає рядок для запису в журнал подій медичного центру."""
        return f"[АУДИТ] Запис №{self.number} (час: {self.time}): змінено стан або створено подію."

class Service:
    """Модель медичної послуги."""

    def __init__(self, name, duration_min):
        self.name = name
        self.duration_min = duration_min

    def show_service(self):
        """Повертає інформацію про послугу."""
        return f"{self.name} ({self.duration_min} хв)"

class Client:
    """Модель пацієнта (клієнта) медичного центру."""

    def __init__(self, full_name, phone):
        self.full_name = full_name
        self.phone = phone

    def show_client(self):
        """Повертає інформацію про клієнта."""
        return f"{self.full_name} (тел: {self.phone})"

class Appointment(AuditableMixin, ABC):
    """Абстрактний базовий клас запису, що успадковує Mixin та контракт."""

    valid_statuses = {"підтверджено", "завершено", "скасовано"}
```

```
        created_count = 0

        def __init__(self, number, time, service, client,
status="підтверджено"):
            self.number = number
            self.time = time
            self.service = service
            self.client = client

            self._status = "підтверджено"
            self.status = status

            Appointment.created_count += 1

        @property
        def status(self):
            """Повертає поточний статус із захищеного внутрішнього
поля."""
            return self._status

        @status.setter
        def status(self, value):
            """Дозволяє встановити лише допустимий статус."""
            if value not in self.valid_statuses:
                raise ValueError(f"Невідомий статус: {value}")
            self._status = value

        @abstractmethod
        def preparation_note(self):
            """Абстрактний метод підготовки до прийому."""
            pass

        def summary(self):
            """Конкретний спільний метод базового класу."""
            return (
                f"Запис №{self.number} | Час: {self.time} | "
                f"Клієнт: {self.client.show_client()} | "
                f"Послуга: {self.service.show_service()} | Статус:
{self.status}"
            )

    class Consultation(Appointment):
        """Похідний клас для консультаційних прийомів лікаря."""

        def __init__(self, number, time, service, client,
doctor_office, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.doctor_office = doctor_office
```

```
        def preparation_note(self):
            """Реалізація абстрактного методу для консультації."""
            return f"Консультація у каб. №{self.doctor_office}:
очікуйте під кабінетом, прийдіть за 10 хв."

    class DiagnosticVisit(Appointment):
        """Похідний клас для діагностичних процедур."""

        def __init__(self, number, time, service, client,
equipment_code, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.equipment_code = equipment_code

        def preparation_note(self):
            """Реалізація абстрактного методу для діагностики."""
            return f"Діагностика (обладнання код:
{self.equipment_code}): дотримуйтеся інструкцій лікаря."

    class AppointmentCalendar:
        """Композиційний компонент: керує розкладом та делегує
операції запису."""

        def __init__(self):
            self.appointments = []

        def add_appointment(self, appointment):
            """Додає запис до внутрішнього сховища календаря."""
            self.appointments.append(appointment)

        def get_schedule_report(self):
            """Делегує завдання формування звіту окремим об'єктам
записів."""
            report = "--- Календар розкладу медичного центру ---\n"
            for app in self.appointments:
                report += f"• {app.summary()} ->
{app.preparation_note()}\n"
            return report

    # Демонстрація роботи
    if __name__ == "__main__":
        # 1. Дослідження порядку вирішення методів (MRO)
        print("=== Дослідження MRO (Method Resolution Order) ===")
        mro_classes = [cls.__name__ for cls in Consultation.mro()]
        print("Порядок MRO для класу Consultation:", " ->
".join(mro_classes))
        print()

        # 2. Створення вигаданих об'єктів
        s1 = Service("Консультація терапевта", 30)
```

```
        s2 = Service("УЗД черевної порожнини", 40)

        c1 = Client("Олександр Т.", "+380501112233")
        c2 = Client("Марія Д.", "+380672223344")

        consultation_app = Consultation(1, "2026-11-04 09:00", s1,
c1, "204", "підтверджено")
        diagnostic_app = DiagnosticVisit(2, "2026-11-04 10:30", s2,
c2, "EQ-9988", "підтверджено")

        # 3. Демонстрація роботи Mixin-методу
        print("=== Демонстрація роботи Mixin (AuditableMixin) ===")
        print(consultation_app.audit_record())
        print(diagnostic_app.audit_record())
        print()

        # 4. Демонстрація композиції та делегування
(AppointmentCalendar)
        print("=== Демонстрація композиції (AppointmentCalendar)
===")
        calendar = AppointmentCalendar()
        calendar.add_appointment(consultation_app)
        calendar.add_appointment(diagnostic_app)

        print(calendar.get_schedule_report())
```

РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

